

DATABRICKS SERIES

WHAT IS DATABRICKS -

Databricks is a unified analytics and data engineering platform that lets you process, and analyze all kinds of data in one place. It's built on top of Apache Spark but adds many enterprise features, including:

Collaborative workspaces for data engineers, data scientists, and analysts
Support for data lakes, data warehouses, and machine learning
Delta Lake technology for reliability, governance, and speed

What is Apache Spark?

Apache Spark is:

- An open-source distributed computing engine for processing large datasets.
- Very fast compared to older frameworks like Hadoop MapReduce.
- Can run batch, streaming, SQL, ML, and graph workloads.
- Requires you to manage clusters, storage, integrations, and scaling manually.

Why Databricks is Different from Just Using Spark

Spark alone = fast distributed processing, but you must manage clusters, storage, and integrations manually.

Databricks = adds a friendly UI, auto-cluster management, collaborative notebooks, optimized connectors, governance tools, ML lifecycle support, and Delta Lake for reliable data.

It's like the difference between owning a powerful engine vs. a fully built sports car — same power, but one is ready to drive.

1. Setup & Infrastructure

Spark Alone → You install, configure, and manage your own clusters (on-premise or cloud).

Databricks → Fully managed — just click to create clusters, no manual setup.

2. Cluster Management

Spark Alone → You must manually scale up/down, monitor, and shut down clusters.

Databricks → Auto-scaling clusters that start/stop based on workload to save cost.

3. Data Storage

Spark Alone → Reads/writes from external storage (S3, HDFS, Azure Blob), but no built-in transactional layer

Databricks → Uses Delta Lake for ACID transactions, schema enforcement, and time travel.

4. Data Governance & Security

Spark Alone → You handle permissions manually at the file system or storage level.

Databricks → Unity Catalog provides centralized data access control, auditing, and compliance.

5. Collaboration

Spark Alone → No built-in multi-user interface — sharing requires exporting scripts or files.

Databricks → Collaborative notebooks where multiple users can

6. Workflow Automation

Spark Alone → Requires external schedulers (Airflow, cron) for automation.

Databricks → Built-in Jobs scheduler with monitoring, retry policies, and alerts.

7. Machine Learning

Spark Alone → MLlib for basic ML, but no built-in experiment tracking or deployment tools.

Databricks → MLflow integrated for ML lifecycle management (tracking, registry, deployment).

9. Optimization

Spark Alone → You manually tune configurations for performance.

Databricks → Auto-optimizations (e.g., Photon engine for faster SQL, auto-tuning).

10. Integrations

Spark Alone → You set up JDBC/ODBC drivers manually for BI tools.

Databricks → Native connectors to Power BI, Tableau, Looker, Snowflake, etc.

What is the Databricks Lakehouse?

The Lakehouse is Databricks' data architecture that combines the best parts of a Data Lake and a Data Warehouse in a single system.

Data Lake strength → Store all kinds of data (structured, semi-structured, unstructured) cheaply in cloud storage.

Data Warehouse strength → Provide fast queries, governance, and reliability.

☛ Databricks Lakehouse merges these into one unified platform using Delta Lake (the secret sauce).

Problems It Solves

1. Data Silos

Traditionally, companies needed a Data Lake for raw data + a Data Warehouse for analytics. That meant two systems, constant ETL pipelines, and duplicate data.

✓ Lakehouse = one platform, no silos.

2. Reliability Issues in Data Lakes

Normal data lakes (S3, ADLS, GCS) don't support:

ACID transactions
Schema enforcement
Version control

This caused corrupted tables and inconsistent data.

✓ Lakehouse (via Delta Lake) adds ACID transactions + schema validation + time travel.

3. Scalability & Cost

Data warehouses are fast but very expensive and limited to structured data.

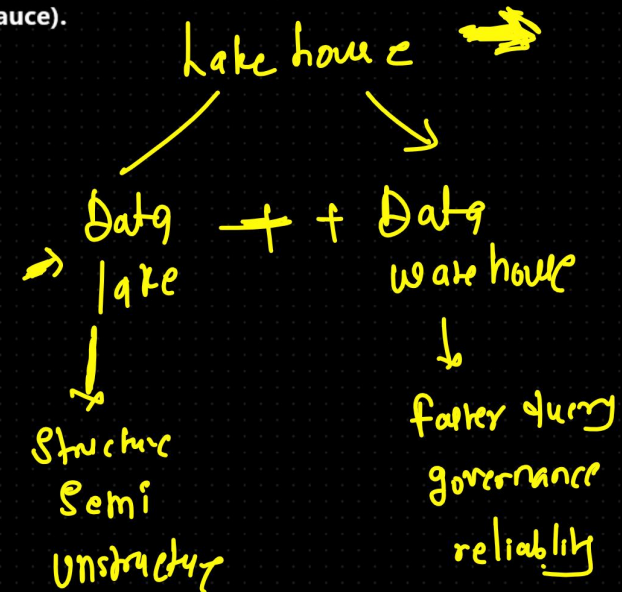
Data lakes are cheap & scalable but slow and unreliable.

✓ Lakehouse keeps data in cheap cloud storage (like a lake) but allows fast SQL queries (like a warehouse).

+ Data lake → Data
Data warehouse → Data

Data + DW

id → int
am → str



⇒ DWH → structure
↓
Tabular
↓
reporting

analytics → slow → ACID → schema, reliable, T → Data lake → structured, semi, un

◆ How It Works Internally



Data Storage

Data is stored in cloud object storage (S3, ADLS, GCS).
Stored in open formats (Parquet, JSON, Avro, ORC).

Delta Lake Layer (the magic ✨)

Adds a transaction log (`_delta_log` folder) on top of files.
Tracks every write/update/delete.
Ensures ACID transactions (no partial writes).
Provides schema enforcement + schema evolution.
Enables time travel (query old versions of data).

Query & Compute Layer

Use Spark, SQL, Python, R, Scala, or Java to process data.
Optimized with Photon Engine (faster SQL queries).
Supports batch, streaming, ML, and BI workloads.
Governance & Security
Unity Catalog manages data access, metadata, and lineage.
Fine-grained security across tables, columns, and files.

◆ Advantages of Databricks Lakehouse

- ✓ Unified Platform – No need for both a lake + warehouse, everything is in one place.
- ✓ Open Data Format – Data stays in open formats (Parquet/Delta), avoiding vendor lock-in.
- ✓ ACID Transactions – Reliable data like a database.
- ✓ Time Travel – Query historical data versions.
- ✓ Performance – Indexing, caching, and Photon engine make queries super fast.
- ✓ Cost-Efficient – Store on cheap cloud storage but query like a warehouse.
- ✓ Supports All Workloads – BI dashboards, ML training, real-time streaming, batch ETL.
- ✓ Scalable & Elastic – Auto-scales with cloud compute.

Cloud → AWS = S3 Data lake
Azure = ADLS Data lake

Delta Layer

Delta layer
+
Data lake

→ Parquet
+ delta-log → ADQ
Versioning
Schemas

What is Unity Catalog? →

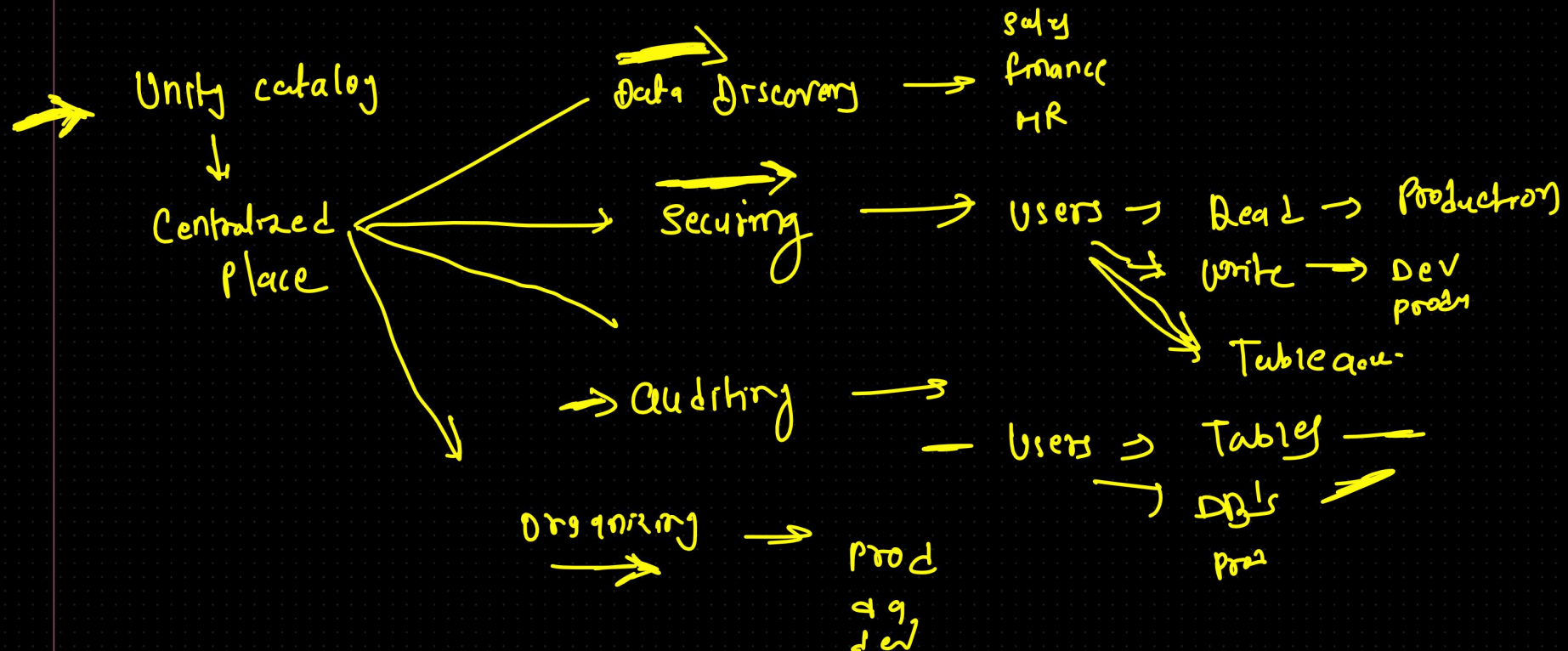
Unity Catalog is Databricks' governance and data management layer for all data and AI assets.

Think of it as a centralized system for organizing, securing, auditing, and discovering data across all workspaces and cloud environments (Azure, AWS, GCP).

It manages:

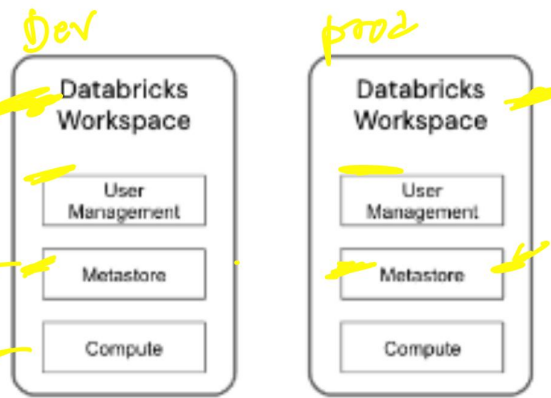
- Tables (structured data, Delta tables)
- Files (unstructured/semi-structured in data lake)
- Machine learning models
- Functions
- Dashboards

☞ In short: One catalog to govern everything (instead of every workspace managing its own metadata & permissions separately).

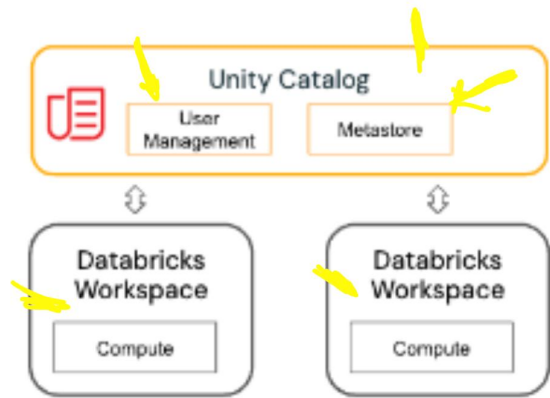


Handwritten arrow pointing to the 'Without Unity Catalog' section.

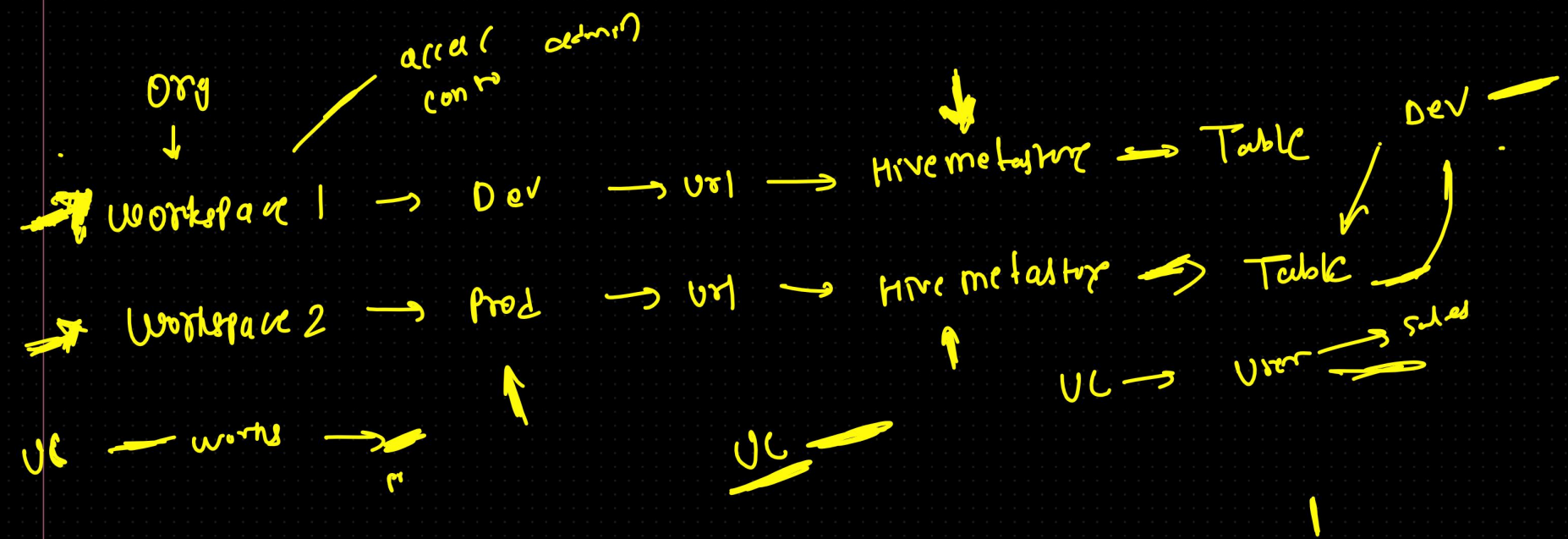
Without Unity Catalog



With Unity Catalog



Handwritten labels 'dev' and 'prod' pointing to the respective workspaces in the 'With Unity Catalog' diagram.



MANAGED VS EXTERNAL Table in Databricks?

In Databricks, tables can be created in two ways:

Managed Table →

External Table →

Managed Table →

Definition

A Managed Table means Databricks manages both the metadata AND the underlying data. Data is stored in Unity Catalog location (or workspace default storage if no UC). Dropping the table also deletes the underlying files.

Characteristics

Data stored in default managed storage location:

UC: <UC managed storage location>/<catalog>/<schema>/<table>

Without UC: /user/hive/warehouse/<table-name>

Databricks controls lifecycle of data (create, drop).

Easier for prototyping / temporary workloads.

You don't specify path while creating the table.

External Table →

Definition

An External Table means Databricks manages only the metadata, but data lives in an external path that you specify. Dropping the table removes only the metadata, NOT the underlying files.

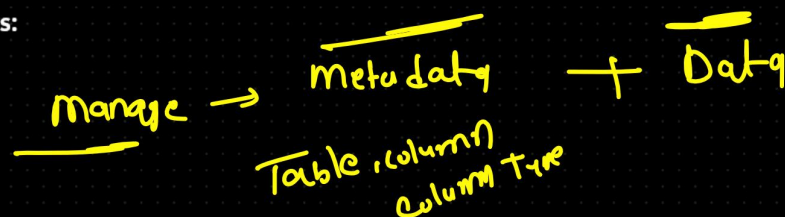
Characteristics

You must specify a LOCATION (path in ADLS/S3/GCS).

Data lifecycle is outside Databricks' control.

Good for shared datasets, compliance, or if you want to keep data even if the table is dropped.

Useful when multiple engines (Databricks, Synapse, Athena, Hive) need to access the same data.



Drop Table

Table schema
Data

-- Managed table (no location specified)

```
CREATE TABLE sales.curated.orders
(
  order_id STRING,
  customer_id STRING,
  amount DECIMAL(10,2),
  order_date DATE
)
USING DELTA;
```

Data

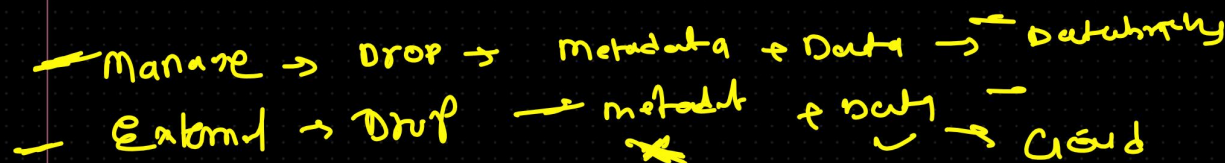
-- External table with explicit path

```
CREATE TABLE
sales.raw.orders_bronze (
  order_id STRING,
  customer_id STRING,
  product_id STRING,
  amount DECIMAL(10,2),
  order_date DATE
)
```

USING DELTA

LOCATION

'abfss://datalake@storageaccount.dfs.core.windows.net/raw/sales/orders/';



Volumes IN DATABRICKS

A Volume in Unity Catalog is:

A logical container to store unstructured or semi-structured data, just like a folder in your data lake.
Governed and integrated into the 3-level namespace (catalog.schema.volume).
Works alongside tables (structured data) and functions/views (logical objects).

Supports both managed and external storage:

Managed Volume → Databricks manages lifecycle; files deleted if volume is dropped.

External Volume → Files remain in external storage even if dropped from UC.

👉 Analogy:

Table = Excel sheet (structured rows & columns).

Volume = Folder with multiple file types (CSVs, PDFs, Images).

Problems Enterprises Faced Before UC Volumes

Problem 1: Data Fragmentation

Files scattered across /raw, /staging, /prod folders.
No central catalog of "where is what."

Problem 2: Security Gaps

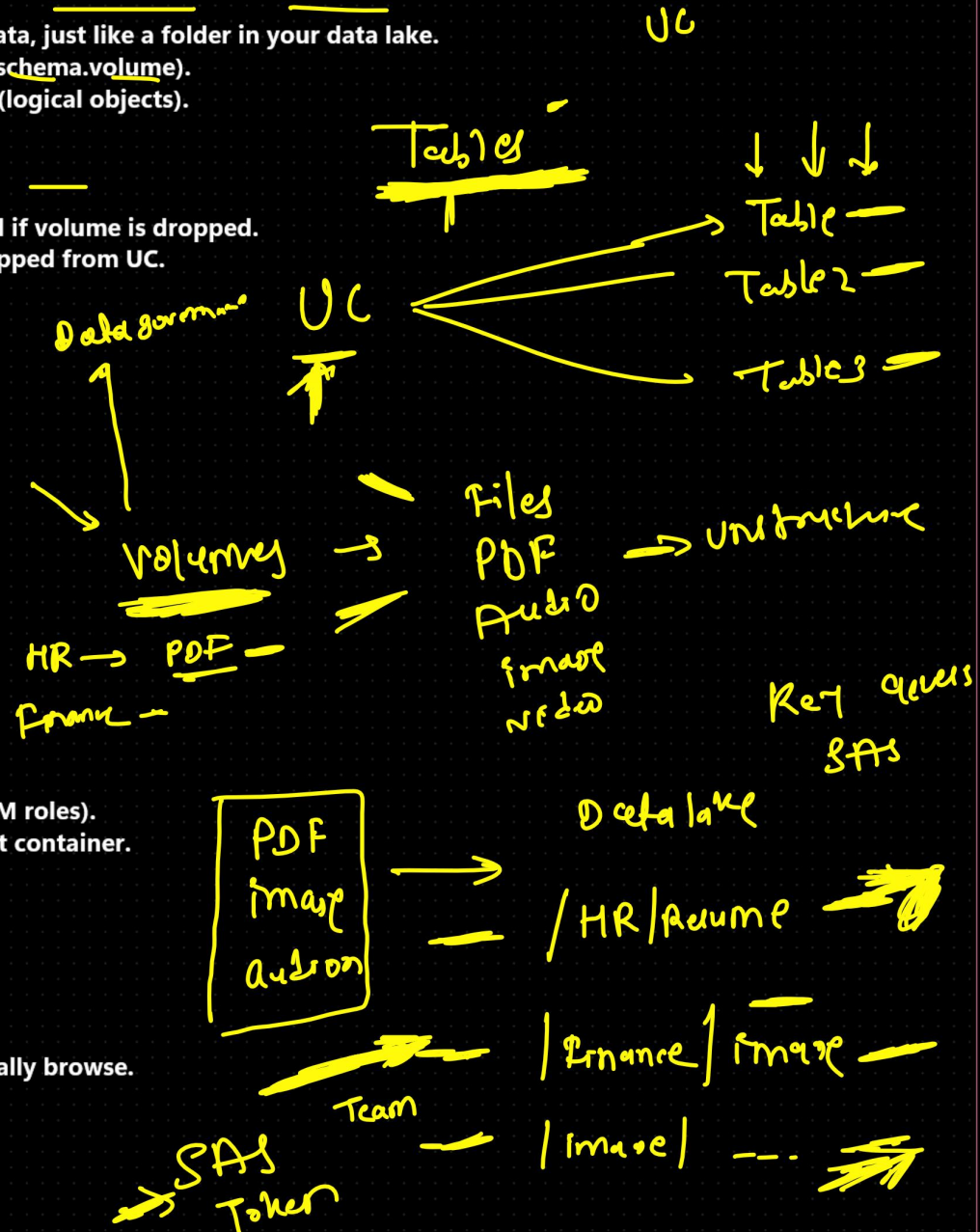
Access control based only on storage credentials (SAS keys, IAM roles).
If a user has a SAS key → they can read/write everything in that container.
Zero visibility into which user accessed which file

Problem 3: Lack of Metadata & Governance

Tables had schemas & metadata.

Files = black holes: no lineage, no owner, no description.

No search functionality: analysts had to "ask around" or manually browse.



How Unity Catalog Volumes Fix These Problems

✓ Centralized Namespace

Files and tables live under the same structure:

catalog.schema.table

catalog.schema.volume

Example:

retail.sales.orders → structured table.

retail.sales.raw_files → volume holding unstructured CSV/JSON/XML.

✓ Fine-Grained Access Control

Permissions applied inside Databricks, not at storage account.

GRANT READ FILES ON VOLUME retail.sales.raw_files TO analyst_group;

GRANT WRITE FILES ON VOLUME retail.sales.raw_files TO engineer_group;

✓ Metadata & Governance for Files

Each volume has owner, description, tags, lineage.

Files inside volumes can be catalogued and discovered.

✓ Unified Audit & Lineage

UC logs file access → "who read which file at what time."

Helps with compliance (GDPR/CCPA).

Advantages of Volumes

Unified Governance → Files are now governed like tables.

Data Discovery → Analysts can find CSV/Parquet directly from UC search.

Improved Security → No raw storage keys; RBAC at file level.

Cross-Workspace Access → No duplication of data/permissions.

Compliance → File access is logged.

Supports Any File → CSV, JSON, parquet, ML model artifacts, PDFs, images, etc.

Lifecycle Management → Managed volumes auto-delete with UC cleanup.

HR → /raw/HR
Eng → /raw/eng

SAS Token

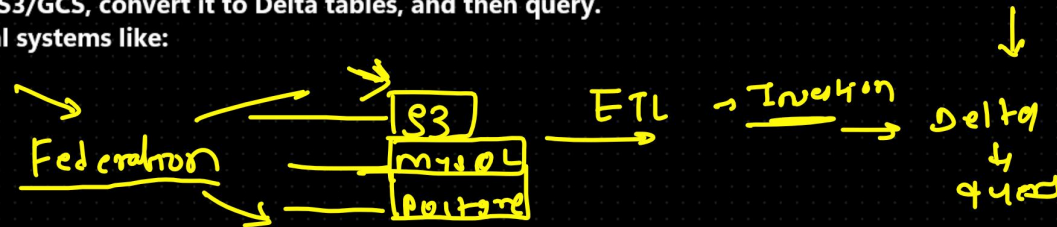
VC

What is Lakehouse Federation in Databricks?

Lakehouse Federation = querying external data sources directly from Databricks SQL or notebooks without copying data into Delta tables.

Normally, in Databricks you ingest raw data into ADLS/S3/GCS, convert it to Delta tables, and then query.
But sometimes, you already have data stored in external systems like:

- Snowflake
- PostgreSQL / MySQL / SQL Server / Oracle
- BigQuery
- AWS Redshift
- SAP, etc.



Instead of duplicating or ETL'ing all this data into Databricks, you can federate → create a link to those systems and query them as if they were Databricks tables.

So your Lakehouse can act as a single pane of glass across multiple systems.

Advantages / Why do we need this?

Lakehouse Federation + Foreign Tables/Catalogs solve real-world integration problems:

- ✓ Avoids Data Duplication – No need to ETL or replicate data from multiple warehouses/databases → directly query in place.
- ✓ Central Governance (Unity Catalog) – Even external data respects Databricks security, RBAC, auditing. This is huge for enterprises.
- ✓ Cross-System Analytics – You can join Delta tables with Snowflake/Postgres tables in a single SQL query.
- ✓ Faster Prototyping – Analysts don't wait for pipelines; they can query existing DBs instantly.
- ✓ Cost Optimization – You don't need to pay for storage + compute in two systems by duplicating data.



What problems does it solve?

Before federation:

You had to write ETL pipelines (ADF, Databricks jobs, Informatica, etc.) to copy data into Databricks.
Data became stale (delays between extraction and ingestion).

Governance was split → Snowflake had its own security, Databricks had its own.

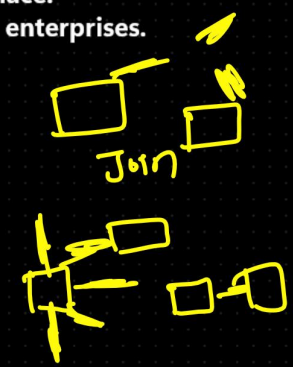
Queries across systems were hard → you'd extract to files and load into Databricks.

Now with Federation:

No extra pipelines.

Near real-time access to external data.

Single governance layer (Unity Catalog policies apply even to external data).



🚀 What is Databricks SQL?

Databricks SQL (DBSQL) is a SQL-first workspace inside Databricks.

It is built for business analysts, BI users, and decision-makers to query data, build dashboards, and connect BI tools (like Power BI, Tableau) — without needing notebooks, Python, or Spark skills.

It runs on SQL Warehouses, which are special compute clusters optimized for fast, concurrent, low-latency SQL queries.

Why do we need DBSQL when we already had Notebooks + Spark SQL?

Notebooks + Spark SQL were designed for data engineers/scientists → good for heavy ETL, ML, or complex pipelines.

But they are not friendly for analysts/business users who just want to write SQL or use dashboards.

Problems with only using notebooks:

→ Hard for non-technical users (need to know Python/Scala, cluster configs).

→ Not great for BI dashboards (Spark clusters are slow for many small, frequent queries).

→ High cost (clusters may stay running, even if idle).

→ No built-in dashboards or alerts.

Limited concurrency (a few notebook queries at once vs 100s of BI users).

👉 So, notebooks are great for creating data but bad for consuming data at scale.

🔧 What problems DBSQL solves

- Performance issue → Dashboards refresh fast with Photon engine + caching.

- Concurrency issue → 100s of users can query at the same time reliably.

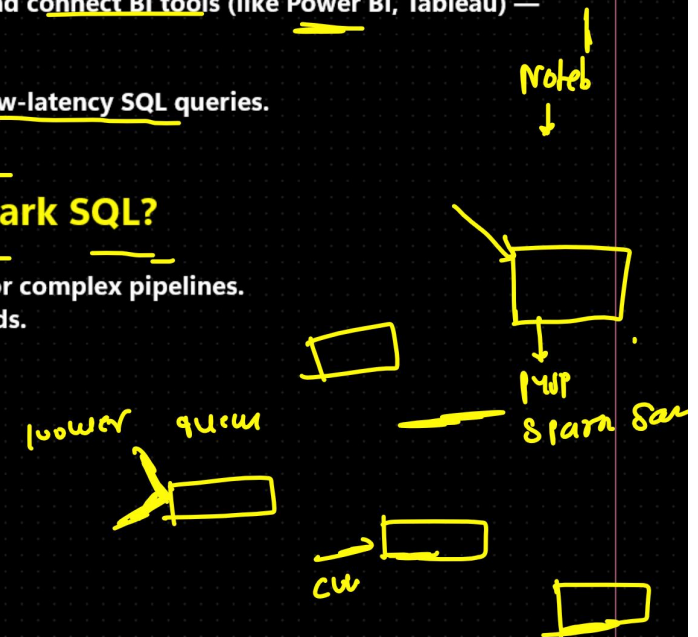
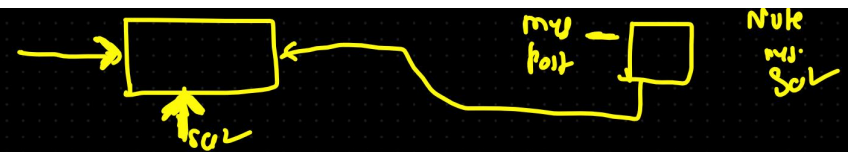
- Cost issue → SQL Warehouses auto-scale & auto-pause, no wasted clusters.

- Usability issue → Analysts get a simple SQL interface, not complex notebooks.

- Governance issue → With Unity Catalog, all queries follow the same security rules (row/column masking, lineage, audit).

- Tool integration issue → Easy to connect with BI tools (Power BI, Tableau, Looker) via SQL endpoints.

- Dashboarding issue → DBSQL has built-in dashboards & alerts, no extra tool required.



Advantages & Business Value

For Analysts/Users

Simple → Just write SQL, no Python needed.
Faster dashboards and queries.
Built-in dashboards, alerts, query history.
Can self-serve without depending on engineers.

For Business

One source of truth → Everyone queries the same Delta data, no copies.
Lower cost → No need for a separate data warehouse like Snowflake/Redshift.
Governance → Security and compliance built-in (important for finance, healthcare, etc.).
Faster decision making → Executives see live dashboards, not stale reports.
Unified platform → Data engineers, data scientists, and business users all work on the same Lakehouse.

What are Lakehouse Jobs in Databricks?

In Databricks, a Job = a way to automate a task (like running a notebook, SQL query, pipeline, or workflow) on a schedule or trigger.

Lakehouse Jobs simply means running and scheduling tasks (jobs) inside the Databricks Lakehouse.

Instead of you manually opening a notebook and clicking "Run", a Job does it automatically, at scale, with monitoring and retry features.



Retail Sales Analytics Project - Databricks + Purview + SparkSQL

Medallion Architecture in Databricks

↑ Dashboard

Business Use Case

A mid-size retail company is struggling to consolidate and analyze its sales transactions that are spread across different sources and formats. The business wants a unified analytics solution to:

- clean messy data (inconsistent formats, missing values, duplicates),
- build a single source of truth for sales, customers, and products,
- monitor KPIs such as revenue, return rate, repeat customers, average order value,
- enable business teams to explore insights across product, category, customer, and time dimensions,
- and use dashboards for data-driven decisions (e.g., inventory, pricing, promotions).

messy

Retail < { CSV → clean, JSON → }

clean

Solution - Medallion architecture

- Bronze Layer: Ingest messy CSV/Parquet files manually (as simulated in Databricks free edition).
- Silver Layer: Clean and standardize data (fix dates, remove \$ from prices, normalize product names, handle missing/negative quantities, remove duplicates).
- Gold Layer: Create one rich aggregate Delta table with grouped metrics across product, customer, category, and time (year, month, date).

KPIs Derived:

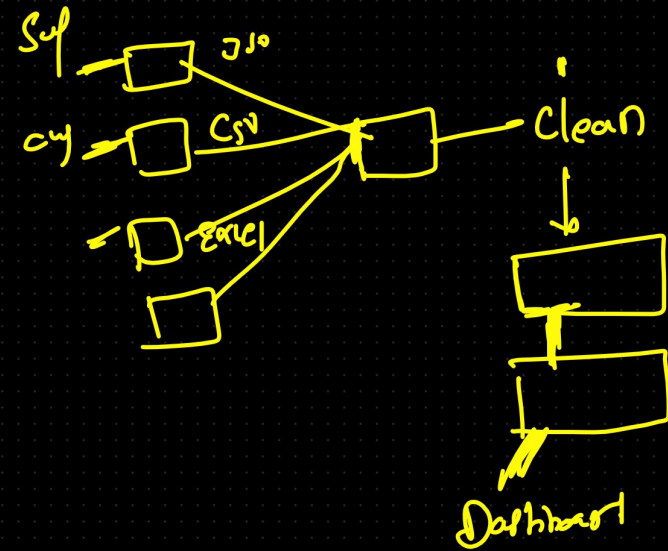
Total Revenue, Total Orders, Total Items Sold
Average Order Value (AOV), Average Price per Item
Repeat Customer Rate, Purchase Frequency
Return Rate (by count & value)
Monthly Sales Trends & MoM Growth
Top Products by Revenue and Quantity
Customer Lifetime Value (CLV) approximation

Dashboard: Build a Databricks SQL Dashboard showing trends, category performance, top products, and customer insights.

Retail — Sales, Product, Customer
↓

Retail → Sales, CSV
↓
→ Customer, JSON

— Unified Dataset → clean → Sales
Transaction
Product
Customer



m

SPARK STREAMING

Spark Streaming is an extension of Apache Spark that allows you to process live data streams in near real-time. It uses micro-batching or continuous processing:

Spark processes them quickly (like every 1 sec, 10 sec, etc.).

Example sources:

Kafka (real-time events)

Cloud storage (new files arriving)

IoT sensors, logs, clickstream, etc.

Batch processing — process data in discrete jobs on a snapshot of data (daily, hourly, ad-hoc). Good when slight delay is fine and you need to process large historical volumes.

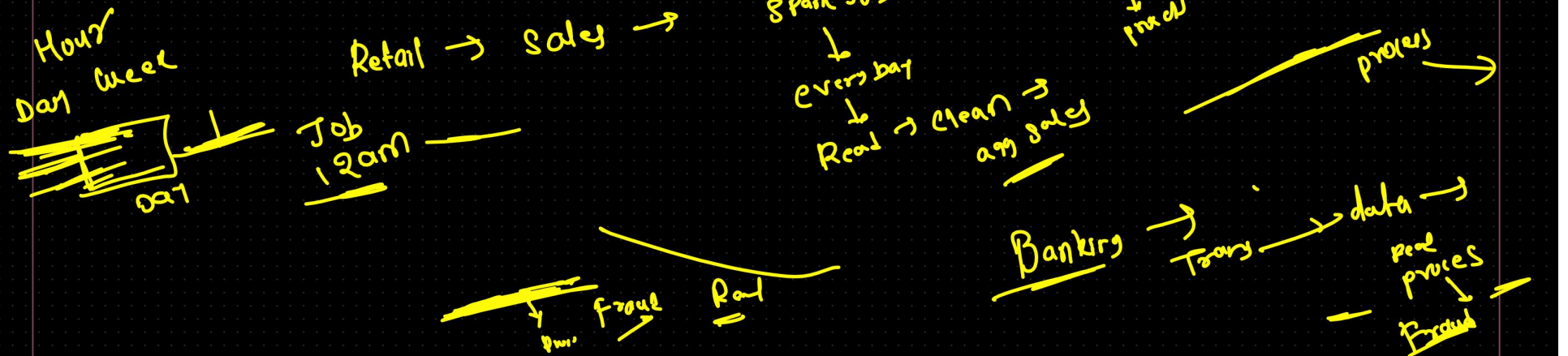
Spark Structured Streaming — continuous processing of incoming events with low latency (micro-batches or continuous execution). Good when you must react quickly to events.

it's used in real-world scenarios like:

Product recommendation → recommend items as users browse (e.g., "Customers also bought...")

Fraud detection → flag suspicious payments, logins, or account takeovers in seconds

Anomaly detection → detect unusual sensor readings, system failures, or traffic spikes



Spark Streaming Recap

In Spark Structured Streaming, you need a source (Kafka, files, socket, etc.) that produces data continuously. Spark then processes it in micro-batches or continuous mode, and writes to a sink (Delta, DB, console, etc.).

👉 Example:

```
df = spark.readStream.format("json").load("/mnt/raw/events")
```

Here, Spark will keep checking the folder for new files.

Problem:

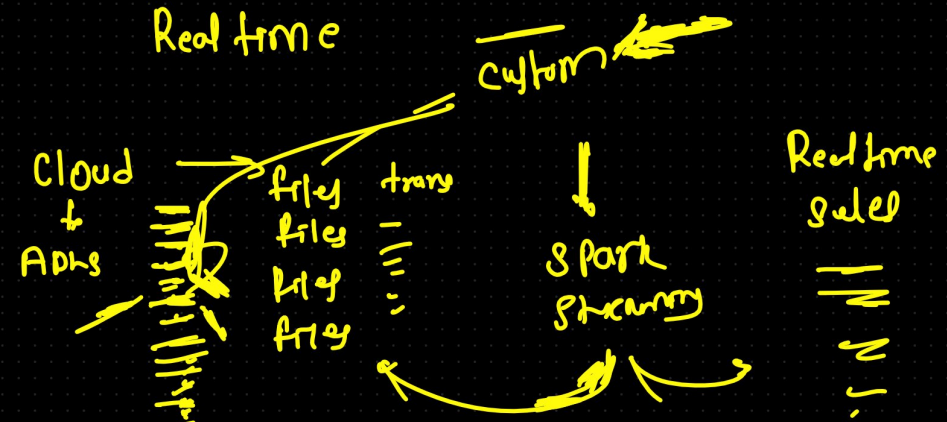
- ➡ It doesn't scale well if you have millions of files.
- ➡ Duplicate/reprocessing issues.
- ➡ Schema changes break the pipeline.

Where Auto Loader Comes In

- Auto Loader is Databricks' optimized source for Spark Streaming when the data comes from cloud storage.

Think of Auto Loader as a specialized streaming source in Spark that:

- Continuously monitors a directory in ADLS/S3/GCS.
- Ingests only new files (incremental).
- Keeps track with checkpoints (so it knows what's already processed).
- Handles schema inference & evolution automatically.
- Scales to billions of files (normal file streaming struggles with this).



AUTO LOADER

Auto Loader is a feature in Databricks that makes it super easy to ingest (load) new data files automatically from cloud storage (like Azure Data Lake, AWS S3, or GCP Storage) into Delta tables.

It's built on top of Spark Structured Streaming.

Instead of you manually telling Spark where new files are, Auto Loader will continuously watch a folder for new data and process it incrementally (only new files).

Why do we need Auto Loader?

Normally in Spark Streaming:

You'd have to write custom logic to detect new files, avoid reprocessing old ones, and handle schema changes.

This is painful and error-prone.

Auto Loader solves this:

1. Incremental processing – Only picks up new files.
2. Schema evolution – Can handle when new columns appear in the data.
3. Scalability – Can process billions of files efficiently.
4. Checkpointing – Keeps track of what's already processed.

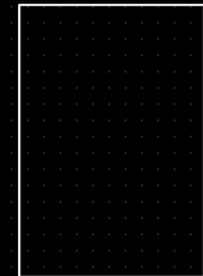
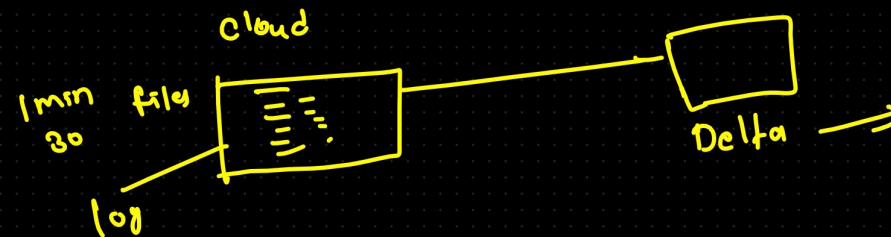
How Auto Loader works (in Spark Streaming terms):

Auto Loader runs as a streaming source in Spark.

It listens to a directory in cloud storage.

Whenever a new file lands, Spark Streaming picks it up and processes it in micro-batches or continuous mode.

It then writes data into Delta Lake (Bronze → Silver → Gold) layers.



◆ Example (Real-time)

Suppose you run an E-commerce website and store all user transactions (JSON files) in Azure Data Lake.

👉 Without Auto Loader:

You'd need to manually rerun jobs, handle duplicates, etc.

👉 With Auto Loader:

As soon as a new transaction file lands in the folder, Auto Loader:

Detects the file

Reads it into a Spark Streaming DataFrame

Cleans/transforms it

Saves it into Delta Table in near real-time

This helps in real-time dashboards for:

Fraud detection 🚨

Live sales monitoring 📊

Recommendation engines 🔄

1. Checkpointing in Auto Loader

In Spark Structured Streaming, a checkpoint is where Spark saves information about:

What files it has already read.

The progress of the stream (so it doesn't reprocess old files).

The state of the query.

Why it matters in Auto Loader?

Without checkpointing, Spark would re-read all files every time you restart the stream.

With checkpointing, Auto Loader knows exactly where it left off.

Schema Management in Auto Loader

Data schema can evolve over time (new columns added, column types changed, etc.). Normally, this breaks ingestion jobs.

Auto Loader has two modes for schema handling:

a) Schema Inference

Auto Loader automatically detects schema from files.

Stores schema in a schema location path (metastore).

Next time new data comes, it compares with the saved schema.

b) Schema Evolution

If a new column appears, Auto Loader can automatically add it.

This prevents ingestion failure when data changes.

Example – Real Scenario

Imagine your e-commerce site logs transactions like this:

Day 1 file (JSON):

```
{"txn_id": 101, "customer_id": 1, "amount": 500}
```

Day 5 file (new schema):

```
{"txn_id": 102, "customer_id": 2, "amount": 700, "payment_method": "UPI"}
```

Without schema evolution → Job fails ❌.

With Auto Loader schema evolution → The new column payment_method gets added automatically ✅.

What is COPY INTO?

COPY INTO is a Databricks SQL command used to load data from files (CSV, JSON, Parquet, etc.) stored in cloud storage (S3, ADLS, GCS) into a Delta table.

It's idempotent → already loaded files are skipped automatically, so you won't insert duplicates accidentally.

Key differences at a glance

Mode

COPY INTO: Batch (you run it on a schedule or ad-hoc).

Auto Loader: Streaming or triggered micro-batches; continuously processes arrivals.

Scale & cost

COPY INTO: Good for thousands of files.

Auto Loader: Built for millions+; fewer listing ops, cheaper & more efficient at scale.

Schema handling

COPY INTO: Can infer input schema and evolve target with mergeSchema. Works even with schemaless target tables.

Auto Loader: Automatic schema inference & evolution, plus rescued_data for unexpected columns/types.

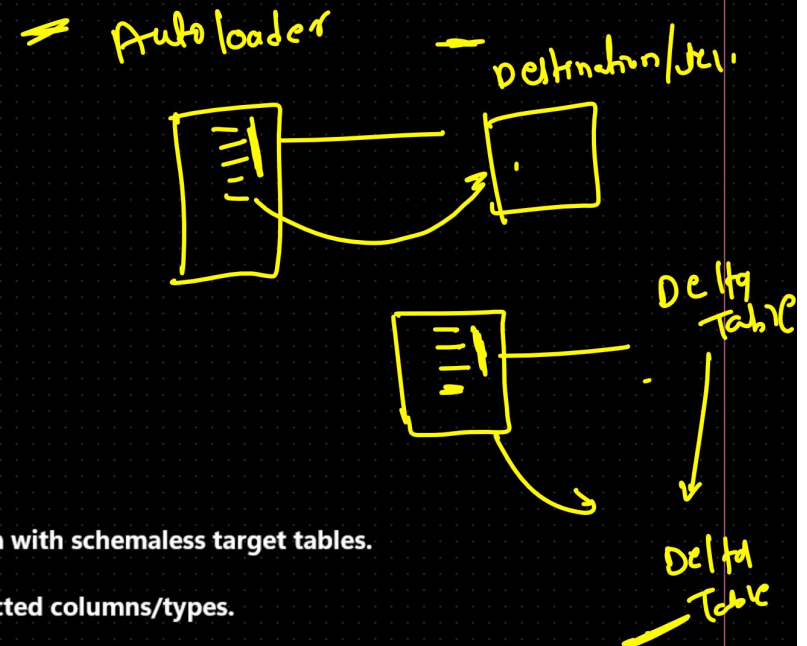
When to use which

Choose COPY INTO if

- You have one-off or scheduled batch loads.
- Volume is modest (up to thousands of files).
- You want pure SQL simplicity (no streaming job to manage).

Choose Auto Loader if

- Files arrive continuously and you need near-real-time ingestion.
- You expect millions+ files over time and want lower listing cost.
- You want built-in schema drift handling and pipeline resilience at scale.



What is DLT (Delta Live Tables)?

Delta Live Tables (DLT) is a framework for building reliable ETL/ELT pipelines on Databricks using SQL or PySpark.

It is declarative, meaning instead of writing step-by-step jobs (imperative), you just declare what data you want and how it should be transformed.

Databricks handles the orchestration, dependency management, monitoring, and error recovery for you.

Think of it like a **modernized version of ETL pipelines** that is:

- Auto-managed (no need to manually schedule and monitor complex workflows)
- Data quality aware (can enforce expectations and rules on data)
- Optimized for Delta Lake (storage format that gives ACID transactions, time travel, schema enforcement, etc.)



DLT
↓
ETL/ELT



SQL/PySpark

Problems DLT Solves

1. Complex Pipeline Orchestration

Traditional ETL = manually chaining notebooks/jobs, with dependencies (bronze → silver → gold).

DLT automatically manages dependencies between tables. If table A depends on B, Databricks ensures B is built first.



Dependency

T
800

Databricks

ETL

Dependency manag

2. Data Quality Issues

Pipelines often break due to bad or missing data.

DLT lets you define data quality rules ("expectations") like:

EXPECT col_x > 0 ON VIOLATION DROP ROW;

This prevents corrupt records from flowing downstream.



Company

Table1
(raw)

Table2
(clean)

Table3
(agg)

3. Scalability & Performance

Instead of managing scaling, caching, and optimizations manually, DLT uses:

- Auto-scaling clusters
- Photon engine for speed
- Delta Lake optimizations (Z-Ordering, caching, schema evolution)



clean

agg

Jobs1

Jobs2

Jobs3

clean

4. Operational Complexity

Usually, ETL teams build lots of jobs for ingestion, cleaning, transformations, and aggregates.

DLT reduces this by:

Defining tables once in SQL/PySpark

Automatically keeping them up-to-date (batch or streaming)

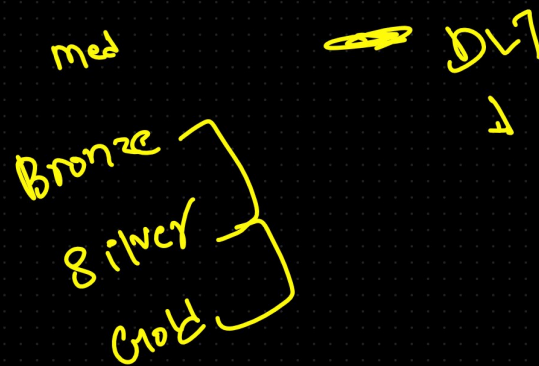
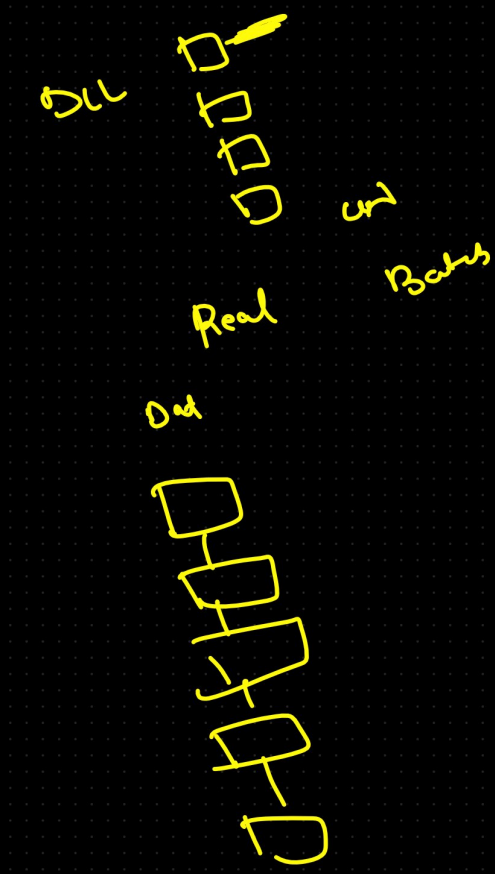
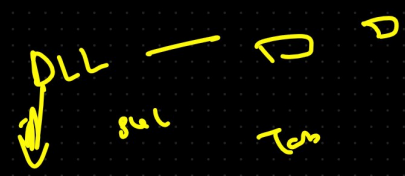
5. Observability & Monitoring

Hard to monitor failures in classic pipelines.

DLT comes with a visual DAG and lineage tracking → you can see how data flows from raw to gold tables.

Advantages of DLT

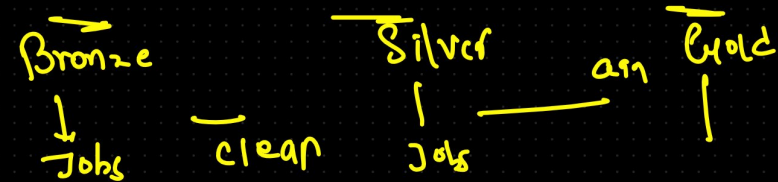
- ✓ Declarative Approach → Write less orchestration logic, focus on transformations.
- ✓ Unified Batch & Streaming → One code works for both real-time (Kafka, Auto Loader) & batch ingestion.
- ✓ Data Quality Rules → Enforce schema, filter out bad records, maintain trustworthy data.
- ✓ Lineage & Monitoring → Visual DAG, job history, data freshness tracking.
- ✓ Auto Management → Scaling, retries, incremental processing handled automatically.
- ✓ Integration with Medallion Architecture → Perfect for Bronze → Silver → Gold layer design.



◆ Key Use Cases of DLT

1. Medallion Architecture Pipelines

Ingest raw data into Bronze (e.g., JSON from Kafka/S3).
Clean/transform into Silver (filtering nulls, joins, normalization).
Aggregate into Gold (reporting, ML features, BI dashboards).
DLT makes these seamless with dependency management.



2. Streaming + Batch Unified

Say you have Kafka streaming transactions + daily batch files.
Instead of two separate pipelines, DLT handles both with one code.

Table Table
Stream Batch

3. Data Quality Enforcement

Finance/Healthcare industries → cannot risk bad records.
Example: Transactions with negative amounts automatically filtered before Silver layer.

Finance

4. Simplify ETL for Data Analysts

Analysts who know only SQL can define DLT tables using CREATE LIVE TABLE without Spark/Scala knowledge.

5. Self-Healing Pipelines

If upstream data changes or jobs fail, DLT can auto-recover and continue from the last checkpoint.

6. Feature Engineering for ML

Build curated Gold tables with time-windowed aggregates and keep them always fresh for ML models.

◆ When to Use DLT vs Traditional Jobs

Use DLT when:

You want declarative, auto-managed, lineage-aware pipelines.

You care about data quality rules & reliability.

You're building streaming + batch unified pipelines.

You want Medallion architecture with fewer moving parts.

Use classic notebooks/jobs when:

You need very custom ETL logic.

You're working with non-Delta formats.

You want one-off data processing (not continuous pipelines).

DLT

Tn

Finance →

Databricks Delta Live Tables (DLT) you'll see Streaming Tables, Materialized Views, and Views.

1. Streaming Table

Definition: A table that processes incremental (streaming) data and maintains a live, continuously updated state.

How it works:

Built on Structured Streaming in Spark.

- Reads new records as they arrive from a source (Kafka, files via Auto Loader, Kinesis, etc.).
- Applies your transformations (filters, joins, aggregations).
- Updates the target table incrementally.

Use case:

When your source data is real-time or append-only (new events, transactions, logs).

Example: Processing clickstream logs, IoT sensor data, financial transactions.

2. Materialized View

Definition: A batch or incremental physical table that stores results of transformations. It is automatically managed (incremental refresh, schema evolution, quality enforcement).

How it works:

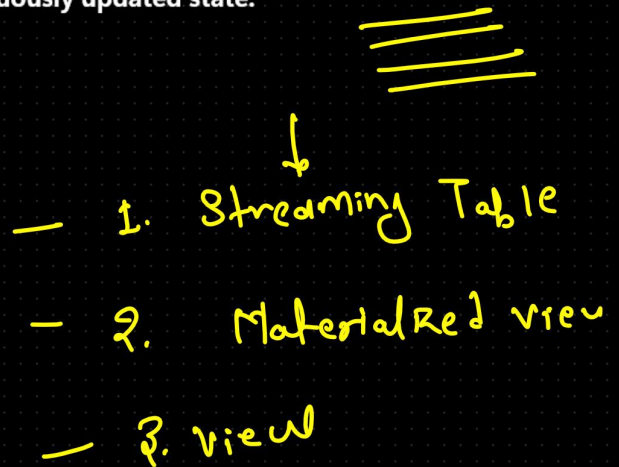
- Stores the output physically in Delta format.
- Optimized for incremental computation: it recomputes only the changes from the source, not the full dataset.
- Provides durability and performance (fast reads for BI tools, aggregations).

Use case:

When you want curated Silver/Gold tables (cleaned and aggregated data).

When data will be queried frequently (Synapse, Power BI, Tableau).

Example: Aggregating daily sales by region from streaming orders.



◆ 3. View

Definition: A logical view (not materialized) of data in the pipeline.
It only exists inside the pipeline for intermediate transformations.

How it works:

Not stored on disk.

Acts like a SQL view in a database: just a query definition.

Good for reusability inside pipelines.

Use case:

For intermediate logic where materialization is unnecessary.

Keeps pipeline simpler without creating unnecessary storage.

Example: Parsing JSON or normalizing columns before writing to a materialized table.

GitHub integration - databricks

GitHub + Databricks

Codes
↓
Central
repo
↓
Github

